Блочное LU-разложение через GAXPY

Курс «Параллельные алгоритмы» · вариант 5

$$A = L\,U$$

Теория · Последовательный алгоритм · Векторизация · Автоматическое распараллеливание · MPI

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

Содержание

Теория LU-разложения

1. Треугольные матрицы
2. Определение LU-разложения
3. Метод Гаусса в матричной форме
4. Поэлементные формулы (схема Дулиттла)
5. Существование и единственность
6. Блочная форма и дополнение Шура
7. Пример: разложение матрицы 3 × 3
8. Трудоёмкость
9. Устойчивость и выбор ведущего элемента
10. Сводка формул

Литература

Этап 1. Последовательное блочное LU-разложение через GAXPY

1. Постановка задачи
2. Операция GAXPY
3. Шесть форм LU и jki-форма (left-looking)
4. Блочная jki-форма
5. Вычислительная сложность
6. Решение системы и проверка
7. Тестовая матрица и измерение погрешности
8. Соответствие коду

Литература

Этап 2. Векторизация блочного LU через GAXPY

1. Что такое векторизация
2. Ядро алгоритма — операция AXPY по строке
3. Сборки и флаги компилятора
4. Модель ожидаемого ускорения
5. Результаты
6. Отличия от этапа 1

Этап 3. OpenMP: блочное LU через GAXPY

1. Модель OpenMP
2. Какие циклы можно распараллелить
3. Модель ускорения
4. Результаты

Этап 4. MPI-версия блочного LU через GAXPY

1. Модель MPI

2. Распределение данных
3. Параллельный алгоритм одного блочного шага
4. Анализ стоимости
5. Замер времени
6. Результаты

Теория LU-разложения

Этот документ содержит математическую основу всех четырёх программ: что такое LU-разложение, как оно получается из метода Гаусса, полный вывод формул, условия существования, блочная форма, трудоёмкость и устойчивость. Реализация алгоритма описана в этапе 1.

Обозначения: $A = (a_{ij}) \in \mathbb{R}^{n\times n}$; индексы в этом документе нумеруются **с единицы**, как принято в математике (в коде — с нуля); e_k — k-й столбец единичной матрицы I; $A_k = A_{1:k,\,1:k}$ — ведущая главная подматрица порядка k.

1. Треугольные матрицы

Матрица L **нижняя треугольная**, если $l_{ij} = 0$ при $i < j$, и **верхняя треугольная** (U), если $u_{ij} = 0$ при $i > j$. Нижняя треугольная матрица с единицами на диагонали называется **унитреугольной** (unit lower triangular).

Для дальнейшего понадобятся четыре свойства.

Свойство 1. *Произведение нижних треугольных матриц — нижняя треугольная матрица.*

Пусть L, M — нижние треугольные. Элемент произведения

$$(LM)_{ij} = \sum_{s=1}^{n} l_{is}\, m_{sj}.$$

Слагаемое ненулевое только при $s \le i$ (иначе $l_{is} = 0$) и $s \ge j$ (иначе $m_{sj} = 0$). При $i < j$ таких s нет, значит $(LM)_{ij} = 0$. Диагональ: $(LM)_{ii} = l_{ii}\, m_{ii}$, поэтому произведение унитреугольных матриц — унитреугольная матрица.

Свойство 2. *Определитель треугольной матрицы равен произведению диагональных элементов:*

$$\det L = \prod_{i=1}^{n} l_{ii}.$$

(Разложение по первой строке для нижней треугольной матрицы оставляет одно слагаемое $l_{11}\det L_{2:n,\,2:n}$, далее по индукции.) В частности, унитреугольная матрица всегда невырожденна: $\det L = 1$.

Свойство 3. *Обратная к невырожденной нижней треугольной матрице — нижняя треугольная; к унитреугольной — унитреугольная.*

Столбец $x = (L^{-1})_{:,j}$ — решение системы $Lx = e_j$. Прямая подстановка даёт $x_1 = \dots = x_{j-1} = 0$ (правая часть в этих строках нулевая) и $x_j = 1/l_{jj}$. Значит, выше диагонали у L^{-1} нули, а на диагонали — $1/l_{jj}$.

Свойство 4. *Треугольные системы решаются за $O(n^2)$ операций.* Система $Ly = b$ решается прямой подстановкой, $Ux = y$ — обратной:

$$y_i = \frac{1}{l_{ii}}\Bigl(b_i - \sum_{j=1}^{i-1} l_{ij}\, y_j\Bigr), \quad i = 1,\dots,n; \qquad x_i = \frac{1}{u_{ii}}\Bigl(y_i - \sum_{j=i+1}^{n} u_{ij}\, x_j\Bigr), \quad i = n,\dots,1.$$

Строка i требует $i-1$ умножений и столько же вычитаний, всего $\sum_{i=1}^{n} 2(i-1) + n = n^2$ флопов.

2. Определение LU-разложения

LU-разложением матрицы A называется представление

$$A = LU,$$

где L — нижняя **унитреугольная**, U — верхняя треугольная матрица. Требование $l_{ii} = 1$ (вариант Дулиттла, Doolittle) делает разложение единственным; в варианте Краута (Crout) единицы ставят на диагональ U.

Зачем оно нужно. Если $A = LU$, то система $Ax = b$ заменяется двумя треугольными:

$$Ax = b \iff L\,(Ux) = b \iff \begin{cases} L\,y = b, \\ U\,x = y. \end{cases}$$

Разложение стоит $\frac{2}{3}n^3$ флопов (см. раздел 8), а каждая треугольная система — n^2. Поэтому для m правых частей с одной матрицей метод выгоден: $\frac{2}{3}n^3 + 2mn^2$ вместо $m \cdot \frac{2}{3}n^3$ при повторном методе Гаусса.

3. Метод Гаусса в матричной форме

Один шаг исключения

Метод Гаусса последовательно обнуляет элементы под диагональю. Обозначим $A^{(1)} = A$. Перед шагом k матрица $A^{(k)}$ уже имеет нули под диагональю в столбцах $1, \dots, k-1$:

$$A^{(k)} = \begin{pmatrix} a_{11}^{(1)} & \cdots & a_{1,k}^{(1)} & \cdots & a_{1n}^{(1)} \\ & \ddots & \vdots & & \vdots \\ & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & \vdots & & \vdots \\ & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{pmatrix}.$$

Элемент $a_{kk}^{(k)}$ называется **ведущим** (pivot). Если $a_{kk}^{(k)} \neq 0$, вычислим **множители**

$$l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \qquad i = k+1, \dots, n,$$

и вычтем из каждой строки $i > k$ строку k, умноженную на l_{ik}:

$$\boxed{a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}\, a_{kj}^{(k)}, \qquad i, j = k+1, \dots, n.}$$

Строки $1, \dots, k$ не меняются, а элементы $a_{ik}^{(k+1)}$, $i > k$, становятся нулями (именно для этого выбран множитель: $a_{ik}^{(k)} - l_{ik} a_{kk}^{(k)} = 0$). После $n-1$ шагов получается верхняя треугольная матрица $U = A^{(n)}$.

Матрица Гаусса

Шаг k — это умножение слева на **матрицу Гаусса** (элементарную нижнюю треугольную матрицу)

$$M_k = I - \ell_k\, e_k^T, \qquad \ell_k = (0, \dots, 0,\ l_{k+1,k},\ \dots,\ l_{nk})^T,$$

у которой в векторе ℓ_k первые k компонент — нули:

$$M_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}.$$

Действительно, $(M_k A^{(k)})_{i,:} = A_{i,:}^{(k)} - l_{ik}\, A_{k,:}^{(k)}$ — это и есть вычитание строк. Итак,

$$A^{(k+1)} = M_k\, A^{(k)}, \qquad U = A^{(n)} = M_{n-1} M_{n-2} \cdots M_1\, A.$$

Обратная к матрице Гаусса

Лемма 1. $M_k^{-1} = I + \ell_k\, e_k^T$.

Доказательство. Перемножим:

$$(I - \ell_k e_k^T)(I + \ell_k e_k^T) = I - \ell_k e_k^T + \ell_k e_k^T - \ell_k\,(e_k^T \ell_k)\, e_k^T = I,$$

так как $e_k^T \ell_k = (\ell_k)_k = 0$ (у ℓ_k нули в первых k позициях). ■

Обращение матрицы Гаусса сводится к смене знака множителей.

Сборка матрицы L

Из $U = M_{n-1}\cdots M_1 A$ получаем

$$A = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}\, U = L\,U, \qquad L = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}.$$

Лемма 2. $L = I + \sum_{k=1}^{n-1} \ell_k\, e_k^T$, то есть

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{pmatrix}.$$

Доказательство. Для $i < j$:

$$(I + \ell_i e_i^T)(I + \ell_j e_j^T) = I + \ell_i e_i^T + \ell_j e_j^T + \ell_i\,(e_i^T \ell_j)\, e_j^T,$$

а $e_i^T \ell_j = (\ell_j)_i = 0$, потому что $i < j$ и у ℓ_j нули в позициях $1, \dots, j$. По индукции произведение $M_1^{-1}\cdots M_{n-1}^{-1}$ равно $I + \sum_k \ell_k e_k^T$. ■

Вывод: L **состоит из множителей метода Гаусса**, записанных под диагональю, а U — это матрица, полученная в конце прямого хода. Никаких дополнительных вычислений для L не нужно: множитель l_{ik} можно хранить на месте обнулённого элемента a_{ik}. Именно так программа хранит L и U в одном массиве.

> Порядок важен: $L = M_1^{-1}\cdots M_{n-1}^{-1}$ получается простым «складыванием» множителей, а $M_{n-1}\cdots M_1 = L^{-1}$ — нет: в этом произведении появляются перекрёстные слагаемые $\ell_j (e_j^T \ell_i) e_i^T \neq 0$ при $j > i$.

4. Поэлементные формулы (схема Дулиттла)

Те же формулы можно получить напрямую, без матриц Гаусса, приравняв элементы обеих частей $A = LU$. Так как $l_{is} = 0$ при $s > i$ и $u_{sj} = 0$ при $s > j$,

$$a_{ij} = \sum_{s=1}^{n} l_{is}\, u_{sj} = \sum_{s=1}^{\min(i,j)} l_{is}\, u_{sj}.$$

Случай $i \le j$ (элемент на диагонали или выше). Тогда $\min(i,j) = i$ и последнее слагаемое — $l_{ii}u_{ij} = u_{ij}$:

$$a_{ij} = \sum_{s=1}^{i-1} l_{is}\, u_{sj} + u_{ij} \quad\Longrightarrow\quad \boxed{u_{ij} = a_{ij} - \sum_{s=1}^{i-1} l_{is}\, u_{sj}, \qquad j = i, \dots, n.}$$

Случай $i > j$ (элемент ниже диагонали). Тогда $\min(i,j) = j$, последнее слагаемое — $l_{ij}u_{jj}$:

$$a_{ij} = \sum_{s=1}^{j-1} l_{is}\,u_{sj} + l_{ij}\,u_{jj} \quad\Longrightarrow\quad \boxed{\,l_{ij} = \frac{1}{u_{jj}}\Bigl(a_{ij} - \sum_{s=1}^{j-1} l_{is}\,u_{sj}\Bigr), \qquad i = j+1,\dots,n.\,}$$

Порядок вычислений

Правые части содержат только элементы L и U с меньшими индексами суммирования, поэтому их можно вычислить последовательно. Классический порядок — чередование строк U и столбцов L:

$$\underbrace{u_{11},\dots,u_{1n}}_{\text{строка 1 } U},\ \underbrace{l_{21},\dots,l_{n1}}_{\text{столбец 1 } L},\ \underbrace{u_{22},\dots,u_{2n}}_{\text{строка 2 } U},\ \underbrace{l_{32},\dots,l_{n2}}_{\text{столбец 2 } L},\ \dots$$

Но возможны и другие порядки — например, **по столбцам** (сначала весь столбец j матрицы U, затем столбец j матрицы L). Именно он используется в программе: это jki-форма, или left-looking-алгоритм (см. этап 1, раздел 3).

Связь с методом Гаусса

Промежуточные элементы метода Гаусса — это частичные суммы тех же формул:

$$a_{ij}^{(k)} = a_{ij} - \sum_{s=1}^{k-1} l_{is}\,u_{sj}.$$

(Доказывается индукцией по k из формулы пересчёта $a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}$ и равенства $a_{kj}^{(k)} = u_{kj}$.) При $k = i$ для $i \le j$ это u_{ij}, а при $k = j$ для $i > j$ — числитель формулы для l_{ij}. Значит, **схема Дулиттла и метод Гаусса выполняют одни и те же арифметические операции**, различается только порядок: в методе Гаусса слагаемые вычитаются «сразу во всю оставшуюся матрицу» (right-looking), в схеме Дулиттла — «накапливаются» при вычислении очередного элемента (left- или up-looking). Поэтому и все шесть форм циклов ijk дают один и тот же результат в точной арифметике.

5. Существование и единственность

Теорема. *Пусть все ведущие главные миноры матрицы A порядков $1,\dots,n-1$ отличны от нуля:*

$$\det A_k \neq 0, \qquad k = 1,\dots,n-1.$$

Тогда LU-разложение с унитреугольной L существует и единственно.

Доказательство (индукция по n). При $n = 1$: $L = (1)$, $U = (a_{11})$. Пусть утверждение верно для порядка $n-1$. Запишем A в окаймлённом виде:

$$A = \begin{pmatrix} A_{n-1} & b \\ c^T & a_{nn} \end{pmatrix}, \qquad L = \begin{pmatrix} L_{n-1} & 0 \\ x^T & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} U_{n-1} & y \\ 0 & u_{nn} \end{pmatrix}.$$

Перемножив блоки, получаем, что $A = LU$ равносильно четырём равенствам:

$$\begin{aligned} A_{n-1} &= L_{n-1}\,U_{n-1}, \\ b &= L_{n-1}\,y, \\ c^T &= x^T\,U_{n-1}, \\ a_{nn} &= x^T y + u_{nn}. \end{aligned}$$

1. Ведущие миноры A_{n-1} порядков $1,\dots,n-2$ — это миноры A, они ненулевые; по предположению индукции $A_{n-1} = L_{n-1}U_{n-1}$ существует и единственно.
2. L_{n-1} унитреугольная, значит невырожденная: $y = L_{n-1}^{-1}b$ определяется однозначно.
3. $\det U_{n-1} = \det L_{n-1}\det U_{n-1} = \det A_{n-1} \neq 0$, значит U_{n-1} невырожденна: $x^T = c^T U_{n-1}^{-1}$ определяется однозначно.

4. $u_{nn} = a_{nn} - x^T y$ определяется однозначно. ■

Заметим, что $\det A_n$ (то есть $\det A$) в условии не участвует: вырожденная матрица тоже может иметь LU-разложение, но тогда $u_{nn} = 0$.

Ведущие элементы через миноры

Из $A_k = L_k U_k$ (ведущие подматрицы перемножаются независимо от остальных блоков) и свойства 2:

$$\det A_k = \det L_k \cdot \det U_k = \prod_{i=1}^{k} u_{ii} \quad \Longrightarrow \quad \boxed{u_{kk} = \frac{\det A_k}{\det A_{k-1}}, \qquad \det A_0 := 1.}$$

Следствия:

- метод Гаусса без перестановок не встречает нулевого ведущего элемента **тогда и только тогда**, когда все ведущие миноры $\det A_1, \ldots, \det A_{n-1}$ ненулевые;
- определитель вычисляется как побочный результат разложения:

$$\det A = \prod_{k=1}^{n} u_{kk}.$$

Пример матрицы без LU-разложения: $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ невырожденна, но $\det A_1 = 0$. Из $a_{11} = l_{11} u_{11} = u_{11} = 0$ и $a_{21} = l_{21} u_{11} = 1$ получаем противоречие $0 = 1$. После перестановки строк разложение существует — см. раздел 9.

6. Блочная форма и дополнение Шура

Один шаг как разложение на блоки

Выделим первую строку и первый столбец:

$$A = \begin{pmatrix} a_{11} & w^T \\ v & B \end{pmatrix}, \qquad v, w \in \mathbb{R}^{n-1},\ B \in \mathbb{R}^{(n-1)\times(n-1)}.$$

При $a_{11} \neq 0$ непосредственно проверяется равенство

$$\begin{pmatrix} a_{11} & w^T \\ v & B \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & S \end{pmatrix}, \qquad \boxed{S = B - \frac{v\, w^T}{a_{11}}.}$$

Матрица S называется **дополнением Шура** элемента a_{11}. Это ровно матрица $A^{(2)}_{2:n,\,2:n}$ после первого шага метода Гаусса: $s_{ij} = b_{ij} - l_{i1} w_j$. Если $S = L_S U_S$, то

$$A = \begin{pmatrix} 1 & 0 \\ v/a_{11} & L_S \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U_S \end{pmatrix},$$

то есть LU-разложение — это **рекурсия**: первая строка U — первая строка A, первый столбец L — v/a_{11}, остальное — разложение дополнения Шура. Обновление $B - vw^T/a_{11}$ — **внешнее произведение** (rank-1 update), это right-looking-форма алгоритма.

Блоки размера r

Тот же приём с блоком A_{11} порядка r:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & U_{22} \end{pmatrix}.$$

Перемножив блоки, получаем четыре уравнения и порядок их решения:

$$
\begin{aligned}
A_{11} &= L_{11}U_{11} && \Rightarrow \text{ неблочное LU блока } r \times r,\\
A_{12} &= L_{11}U_{12} && \Rightarrow\ U_{12} = L_{11}^{-1}A_{12} \quad \text{(треугольная система)},\\
A_{21} &= L_{21}U_{11} && \Rightarrow\ L_{21} = A_{21}U_{11}^{-1} \quad \text{(треугольная система)},\\
A_{22} &= L_{21}U_{12} + L_{22}U_{22} && \Rightarrow\ L_{22}U_{22} = \underbrace{A_{22} - L_{21}U_{12}}_{\text{дополнение Шура}}.
\end{aligned}
$$

Последняя строка — рекурсия для матрицы порядка $n - r$. Основная работа — матричное умножение $L_{21}U_{12}$, которое хорошо использует кэш и векторные инструкции. В этом и состоит смысл блочных алгоритмов.

Right-looking и left-looking

Блочная схема выше — **right-looking**: после разложения блочного столбца дополнение Шура сразу пересчитывается для **всей** правой нижней подматрицы. В программах используется **left-looking**-вариант (jki): блочный столбец K не трогается до своей очереди, а затем одним действием получает все накопленные обновления от столбцов левее:

$$
A_{k:n,\,K} - L_{k:n,\,1:k}\,U_{1:k,\,K} = A_{k:n,\,K} - \sum_{\text{блоки } J<K} L_{k:n,\,J}\,U_{J,\,K}.
$$

Сумма справа — это ровно все обновления right-looking-схемы, которые пришлись бы на столбцы K, только применённые одним блочным GAXPY. Арифметика та же, отличается порядок обращений к памяти: left-looking пишет только в текущий блочный столбец и читает всё остальное, right-looking пишет во всю оставшуюся подматрицу.

7. Пример: разложение матрицы 3 × 3

Разложим

$$
A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}.
$$

Методом Гаусса

Шаг 1. Ведущий элемент $a_{11} = 2$. Множители: $l_{21} = 4/2 = 2$, $l_{31} = 8/2 = 4$. Из строки 2 вычитаем удвоенную строку 1, из строки 3 — учетверённую:

$$
A^{(2)} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 3-2\cdot 1 & 3-2\cdot 1 \\ 0 & 7-4\cdot 1 & 9-4\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{pmatrix}.
$$

Шаг 2. Ведущий элемент $a_{22}^{(2)} = 1$. Множитель $l_{32} = 3/1 = 3$:

$$
A^{(3)} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 5-3\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} = U.
$$

Множители образуют L:

$$
L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}.
$$

По формулам Дулиттла

$$
\begin{aligned}
&\text{строка } 1\ U: && u_{11}=2, \quad u_{12}=1, \quad u_{13}=1;\\
&\text{столбец } 1\ L: && l_{21}=\tfrac{4}{2}=2, \quad l_{31}=\tfrac{8}{2}=4;\\
&\text{строка } 2\ U: && u_{22}=3-l_{21}u_{12}=3-2=1, \quad u_{23}=3-l_{21}u_{13}=1;\\
&\text{столбец } 2\ L: && l_{32}=\tfrac{1}{u_{22}}\,(7-l_{31}u_{12})=7-4=3;\\
&\text{строка } 3\ U: && u_{33}=9-l_{31}u_{13}-l_{32}u_{23}=9-4-3=2.
\end{aligned}
$$

Результат тот же.

Проверка

$$
LU=\begin{pmatrix} 1\cdot 2 & 1\cdot 1 & 1\cdot 1 \\ 2\cdot 2 & 2\cdot 1+1 & 2\cdot 1+1 \\ 4\cdot 2 & 4\cdot 1+3\cdot 1 & 4\cdot 1+3\cdot 1+2 \end{pmatrix}=\begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}=A.\ \checkmark
$$

Ведущие элементы через миноры: $\det A_1=2$, $\det A_2=2\cdot 3-1\cdot 4=2$, $\det A=2\cdot 1\cdot 2=4$, откуда $u_{11}=2$, $u_{22}=2/2=1$, $u_{33}=4/2=2$. $\checkmark$

Решение системы

Возьмём $b=A\,(1,1,1)^T=(4,10,24)^T$ — так же строится правая часть в программах.

Прямой ход, $Ly=b$:

$$
y_1=4, \qquad y_2=10-2\cdot 4=2, \qquad y_3=24-4\cdot 4-3\cdot 2=2.
$$

Обратный ход, $Ux=y$:

$$
x_3=\frac{2}{2}=1, \qquad x_2=\frac{2-1\cdot 1}{1}=1, \qquad x_1=\frac{4-1\cdot 1-1\cdot 1}{2}=1.
$$

Получено точное решение $x=(1,1,1)^T$.

8. Трудоёмкость

Считаем отдельно умножения, сложения/вычитания и деления — каждое как один флоп.

На шаге k метода Гаусса ($k=1,\dots,n-1$) обрабатывается подматрица порядка $m=n-k$:

- m делений — множители l_{ik};
- m^2 умножений и m^2 вычитаний — пересчёт $a_{ij}^{(k+1)}$.

Итого

$$
\sum_{k=1}^{n-1}\bigl(m+2m^2\bigr)\Big|_{m=n-k}=\sum_{m=1}^{n-1}m+2\sum_{m=1}^{n-1}m^2=\frac{n(n-1)}{2}+2\cdot\frac{(n-1)\,n\,(2n-1)}{6}.
$$

Упростим:

$$
\frac{n(n-1)}{2}+\frac{n(n-1)(2n-1)}{3}=\frac{n(n-1)\bigl(3+2(2n-1)\bigr)}{6}=\frac{n(n-1)(4n+1)}{6}=\boxed{\frac{2}{3}n^3-\frac{1}{2}n^2-\frac{1}{6}n.}
$$

Главный член — $\frac{2}{3}n^3$. Все формы алгоритма (Гаусс, Дулиттл, шесть порядков циклов, блочные варианты) выполняют одни и те же операции, поэтому имеют одну и ту же трудоёмкость.

Задача	Флопы
LU-разложение	$\frac{2}{3}n^3 + O(n^2)$
решение $Ly = b$ и $Ux = y$	$2n^2$
решение $Ax = b$ с m правыми частями	$\frac{2}{3}n^3 + 2mn^2$
определитель $\det A = \prod u_{kk}$	$\frac{2}{3}n^3 + n$
обратная матрица (n правых частей e_j)	$\frac{2}{3}n^3 + 2n^3 = \frac{8}{3}n^3$ (с учётом нулей в e_j — $2n^3$)

Память: L и U помещаются на место A — n^2 чисел, дополнительная память не нужна.

9. Устойчивость и выбор ведущего элемента

Малый ведущий элемент

Даже если ведущий элемент не равен нулю, он может быть **мал**, и тогда метод неустойчив. Классический пример ($\varepsilon \ll 1$):

$$A = \begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1/\varepsilon & 1 \end{pmatrix} \begin{pmatrix} \varepsilon & 1 \\ 0 & 1 - 1/\varepsilon \end{pmatrix}.$$

При $\varepsilon = 10^{-20}$ в арифметике double $1 - 10^{20}$ округляется до -10^{20}, и вычисленное произведение $\hat{L}\hat{U} = \begin{pmatrix} \varepsilon & 1 \\ 1 & 0 \end{pmatrix}$ — элемент $a_{22} = 1$ полностью потерян. Причина — огромный множитель $l_{21} = 1/\varepsilon$: элементы L и U во много раз больше элементов A.

Частичный выбор ведущего элемента

На шаге k строку с наибольшим по модулю элементом столбца k переставляют на место ведущей. Перестановки накапливаются в матрице перестановок P:

$$PA = LU, \qquad |l_{ij}| \le 1.$$

Такое разложение существует для **любой** невырожденной матрицы. Устойчивость характеризует **коэффициент роста**

$$\rho = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|}.$$

Обратная ошибка метода Гаусса пропорциональна $\rho\, n \varepsilon$ (Higham, гл. 9): при малом ρ метод устойчив. С частичным выбором $\rho \le 2^{n-1}$, но на практике ρ почти всегда мал.

Когда выбор ведущего элемента не нужен

Теорема. *Если A имеет строгое диагональное преобладание по строкам,*

$$|a_{ii}| > \sum_{j \ne i} |a_{ij}|, \qquad i = 1, \dots, n,$$

то метод Гаусса без перестановок выполним (все ведущие элементы ненулевые).

Доказательство. Ведущий элемент $a_{11} \ne 0$, так как $|a_{11}| > 0$. Покажем, что дополнение Шура $S = B - vw^T/a_{11}$ (обозначения раздела 6) снова имеет строгое диагональное преобладание; дальше — индукция.

Для строки i матрицы B (строка $i+1$ матрицы A) условие преобладания: $|b_{ii}| > |v_i| + \sum_{j \ne i} |b_{ij}|$. Для первой строки A: $\sum_j |w_j| < |a_{11}|$. Оценим строку i матрицы S:

$$|s_{ii}| \ge |b_{ii}| - \frac{|v_i|\,|w_i|}{|a_{11}|}, \qquad \sum_{j\ne i}|s_{ij}| \le \sum_{j\ne i}|b_{ij}| + \frac{|v_i|}{|a_{11}|}\sum_{j\ne i}|w_j|.$$

Вычитая,

$$|s_{ii}| - \sum_{j\ne i}|s_{ij}| \;\ge\; \underbrace{|b_{ii}| - \sum_{j\ne i}|b_{ij}|}_{>\,|v_i|} \;-\; |v_i|\,\underbrace{\frac{\sum_j |w_j|}{|a_{11}|}}_{<\,1} \;>\; |v_i| - |v_i| = 0. \qquad \blacksquare$$

Аналогичное утверждение верно для преобладания по столбцам (тогда ещё и $|l_{ij}| \le 1$) и для симметричных положительно определённых матриц (там вместо LU обычно используют разложение Холецкого $A = LL^T$).

Применение к программам. Тестовая матрица строится как $a_{ij} \in [-\frac{1}{2}, \frac{1}{2}]$ плюс $\frac{n}{4}$ на диагонали. Диагональ примерно равна **средней** сумме модулей внедиагональных элементов строки ($\frac{n-1}{4}$), поэтому строгое преобладание выполняется не для всех строк и теорема формально не применима. Тем не менее матрица далека от вырожденной, ведущие элементы остаются порядка $\frac{n}{4}$, и разложение без перестановок даёт ошибки порядка 10^{-14} (см. этап 1, раздел 7).

Оценка обратной ошибки

Для вычисленных в арифметике с плавающей точкой множителей $\hat L, \hat U$ справедливо (Higham, теорема 9.3)

$$\hat L\hat U = A + \Delta A, \qquad |\Delta A| \le \gamma_n\,|\hat L|\,|\hat U|, \qquad \gamma_n = \frac{n\varepsilon}{1 - n\varepsilon},$$

где модуль и неравенство понимаются поэлементно, $\varepsilon \approx 1{,}1 \cdot 10^{-16}$ — машинная точность double. Если элементы $\hat L$ и $\hat U$ не намного больше элементов A (малый рост ρ), разложение **обратно устойчиво**: вычисленные множители — точное разложение близкой матрицы. Для решения системы отсюда следует

$$\frac{\|\hat x - x\|}{\|x\|} \lesssim \kappa(A)\cdot c\,n\,\rho\,\varepsilon, \qquad \kappa(A) = \|A\|\,\|A^{-1}\|,$$

то есть погрешность растёт с размерностью n и числом обусловленности. Именно этот рост измеряется на этапе 1.

10. Сводка формул

Что	Формула
разложение	$A = LU,\ l_{ii} = 1$
шаг Гаусса	$l_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)},\ a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}$
матрица Гаусса	$M_k = I - \ell_k e_k^T,\ M_k^{-1} = I + \ell_k e_k^T$
матрица L	$L = I + \sum_k \ell_k e_k^T$
элементы U	$u_{ij} = a_{ij} - \sum_{s<i} l_{is}u_{sj},\ i \le j$
элементы L	$l_{ij} = \big(a_{ij} - \sum_{s<j} l_{is}u_{sj}\big)/u_{jj},\ i > j$
существование	$\det A_k \ne 0, k = 1,\dots,n-1$
ведущие элементы	$u_{kk} = \det A_k / \det A_{k-1}$
определитель	$\det A = \prod_k u_{kk}$
дополнение Шура	$S = A_{22} - A_{21}A_{11}^{-1}A_{12} = A_{22} - L_{21}U_{12}$
трудоёмкость	$\frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n$
обратная ошибка	$\hat L\hat U = A + \Delta A,\ \|\Delta A\| \le \gamma_n \|\hat L\|\|\hat U\|$

Литература

1. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — §3.2 (LU-разложение, матрицы Гаусса), §3.4 (выбор ведущего элемента).
2. Higham N. J. Accuracy and Stability of Numerical Algorithms. — гл. 9.
3. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем.

Этап 1. Последовательное блочное LU-разложение через GAXPY

Программа: stage1_seq.c. Результаты: stage1_seq.csv, график stage1_error.png.

Этот файл описывает сам алгоритм. Все остальные этапы (векторизация, OpenMP, MPI) выполняют те же вычисления, но по-разному организуют данные и работу.

Математическая теория LU-разложения с полным выводом формул (метод Гаусса в матричной форме, схема Дулиттла, существование и единственность, дополнение Шура, трудоёмкость, устойчивость, разобранный пример) — в отдельном документе.

1. Постановка задачи

Дана квадратная матрица $A \in \mathbb{R}^{n \times n}$. Требуется найти разложение

$$A = L\,U,$$

где L — нижняя треугольная матрица с единичной диагональю ($l_{ii} = 1$), U — верхняя треугольная матрица:

$$L = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{pmatrix}.$$

После разложения система $Ax = b$ решается двумя треугольными системами:

$$L\,y = b \quad \text{(прямой ход)}, \qquad U\,x = y \quad \text{(обратный ход)}.$$

Когда разложение существует

Доказательство этого и следующих утверждений — в теории, раздел 5.

LU-разложение без перестановок строк существует и единственно, если все ведущие главные миноры отличны от нуля:

$$\det A_{1:k,\,1:k} \neq 0, \qquad k = 1, \ldots, n-1.$$

Достаточное условие — строгое диагональное преобладание по строкам или по столбцам, а также симметричная положительная определённость. В программе выбора ведущего элемента (pivoting) **нет**, поэтому тестовая матрица подбирается так, чтобы диагональ была заметно больше остальных элементов (см. раздел 7).

Хранение на месте

Матрицы L и U записываются поверх A. Единичная диагональ L не хранится:

$$A \;\longrightarrow\; \begin{pmatrix} u_{11} & u_{12} & u_{13} & \cdots \\ l_{21} & u_{22} & u_{23} & \cdots \\ l_{31} & l_{32} & u_{33} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

Матрица хранится по строкам (row-major): элемент a_{ij} лежит по адресу `A[i*n + j]`; в коде это макрос `a(i, j)`. Индексы в коде начинаются с нуля, в формулах — для удобства тоже с нуля, если не оговорено иное.

2. Операция GAXPY

GAXPY (generalized Ax plus y) — операция «матрица на вектор плюс вектор»:

$$y \leftarrow y + A\,x, \qquad A \in \mathbb{R}^{m\times k},\ x \in \mathbb{R}^{k},\ y \in \mathbb{R}^{m}.$$

Её можно записать как последовательность операций SAXPY ($y \leftarrow y + \alpha x$) над столбцами A:

$$y \leftarrow y + \sum_{j=1}^{k} x_j\, a_{:,j}.$$

Блочный вариант — та же операция, где вместо вектора x стоит матрица: $Y \leftarrow Y - A\,X$. Именно такая операция составляет основную работу алгоритма.

3. Шесть форм LU и jki-форма (left-looking)

Классический алгоритм Гаусса — это тройной цикл по i, j, k. Шесть перестановок этих индексов дают шесть математически эквивалентных алгоритмов, которые отличаются порядком доступа к памяти (Голуб, Ван Лоун, «Матричные вычисления», §3.2):

Форма	Внутренняя операция	Название
kij, kji	внешнее произведение, обновление всей правой нижней подматрицы	right-looking
ijk, ikj	скалярное произведение / обновление строк	up-looking
jik, jki	**GAXPY** над столбцом	**left-looking**

В jki-форме столбцы L и U вычисляются по одному, слева направо. Столбец j не изменяется до момента, когда до него дойдёт очередь, — тогда он «смотрит влево» на уже готовые столбцы L и обновляется ими.

Вывод jki-формы

Запишем j-й столбец равенства $A = LU$:

$$a_{:,j} = L\,u_{:,j} = \sum_{s=0}^{j} l_{:,s}\,u_{sj},$$

так как $u_{sj} = 0$ при $s > j$. Разделим строки на «верхние» ($i \le j$) и «нижние» ($i > j$):

- верхняя часть, $i \le j$: $a_{ij} = \sum_{s=0}^{i-1} l_{is}u_{sj} + u_{ij}$ — отсюда находится u_{ij} (это прямая подстановка с матрицей $L_{0:j,\,0:j}$);
- нижняя часть, $i > j$: $a_{ij} = \sum_{s=0}^{j-1} l_{is}u_{sj} + l_{ij}u_{jj}$ — отсюда

$$l_{ij} = \frac{1}{u_{jj}}\Big(a_{ij} - \sum_{s=0}^{j-1} l_{is}u_{sj}\Big), \qquad i > j.$$

Обе части вычисляются одной и той же операцией: из столбца $a_{:,j}$ последовательно вычитаются столбцы $l_{:,s}$, умноженные на u_{sj}, — это и есть GAXPY.

Неблочный алгоритм (jki)

```
for j = 0 .. n-1:
    for s = 0 .. j-1:                    # GAXPY: a(:,j) -= l(:,s) * u(s,j)
        for i = s+1 .. n-1:
            a(i,j) -= a(i,s) * a(s,j)
    for i = j+1 .. n-1:                  # деление на ведущий элемент
        a(i,j) /= a(j,j)
```

После шага s элемент a_{sj} уже окончательный и равен u_{sj} (он был обновлён всеми предыдущими столбцами). Самый внутренний цикл идёт по i, то есть **по столбцу** — при хранении по строкам это доступ с шагом n, что плохо для кэша. Это одна из причин перехода к блочной форме.

4. Блочная jki-форма

Разобьём столбцы на блоки ширины r (в программе $r = 32$, константа `R`). Рассмотрим очередной блочный столбец

$$K = [k,\ k+w), \qquad w = \min(r,\ n-k),$$

(последний блок может быть уже r). Все столбцы левее k уже разложены: известны $L_{:,\,0:k}$ и $U_{0:k,\,0:k}$. Запишем равенство $A = LU$ для столбцов K в блочном виде, разделив строки на верхние $0{:}k$ и нижние $k{:}n$:

$$\begin{pmatrix} A_{0:k,\,K} \\ A_{k:n,\,K} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{0:k,\,K} \\ U_{K,\,K} \end{pmatrix},$$

где $L_{11} = L_{0:k,\,0:k}$ (известна), $L_{21} = L_{k:n,\,0:k}$ (известна), $L_{22} = L_{k:n,\,K}$ и $U_{K,K}$ — неизвестные части текущего блока. Отсюда получаются два уравнения.

Шаг 1. Треугольное решение (TRSM)

Верхнее уравнение:

$$L_{11}\,U_{0:k,\,K} = A_{0:k,\,K} \quad\Longrightarrow\quad U_{0:k,\,K} = L_{11}^{-1} A_{0:k,\,K}.$$

Это треугольная система с w правыми частями. Обратная матрица не вычисляется — система решается прямой подстановкой по строкам:

$$u_{i,K} = a_{i,K} - \sum_{j=0}^{i-1} l_{ij}\,u_{j,K}, \qquad i = 1, \dots, k-1,$$

где $u_{i,K}$ — строка длины w. Каждая операция $u_{i,K} \mathrel{-}= l_{ij}\,u_{j,K}$ — это SAXPY над **строкой** длины w, которая лежит в памяти подряд.

Шаг 2. Блочный GAXPY

Нижнее уравнение:

$$A_{k:n,\,K} = L_{21}\,U_{0:k,\,K} + L_{22}\,U_{K,K}.$$

Перенесём известное слагаемое влево:

$$\boxed{\tilde{A}_{k:n,\,K} = A_{k:n,\,K} - L_{k:n,\,0:k}\,U_{0:k,\,K}}$$

Это матричная операция $Y \leftarrow Y - LX$, то есть блочный GAXPY. Поэлементно для каждой строки $i \ge k$:

$$\tilde{a}_{i,K} = a_{i,K} - \sum_{j=0}^{k-1} l_{ij}\, u_{j,K}.$$

На эту операцию и на шаг 1 приходится почти вся арифметика (см. раздел 5).

Шаг 3. LU панели

Остаётся равенство

$$\tilde{A}_{k:n,\,K} = L_{22}\, U_{K,K},$$

то есть обычное (неблочное) LU-разложение «высокой узкой» матрицы размера $(n-k) \times w$ — **панели**. Оно выполняется неблочной jki-формой из раздела 3, только внутри столбцов K:

$$\tilde{a}_{i,c} \mathrel{-}= \tilde{a}_{i,j}\, \tilde{a}_{j,c}, \quad j \in [k, c),\ i > j; \qquad l_{ic} = \tilde{a}_{ic} / u_{cc}, \quad i > c.$$

Полный алгоритм

```
for k = 0, r, 2r, ... < n:
    w = min(r, n - k);  K = [k, k+w)

    # 1) U(0:k, K) = L11^{-1} A(0:k, K)
    for i = 1 .. k-1:
        for j = 0 .. i-1:
            A(i, K) -= A(i, j) * A(j, K)

    # 2) GAXPY: A(k:n, K) -= L(k:n, 0:k) * U(0:k, K)
    for i = k .. n-1:
        for j = 0 .. k-1:
            A(i, K) -= A(i, j) * A(j, K)

    # 3) неблочное jki-LU панели A(k:n, K)
    for c = k .. k+w-1:
        for j = k .. c-1:
            A(j+1:n, c) -= A(j+1:n, j) * A(j, c)
        A(c+1:n, c) /= A(c, c)
```

Это ровно функция `block_lu_gaxpy` в stage1_seq.c. Шаги 1 и 2 в коде имеют одинаковое тело `a(i, c) -= l * a(j, c)` и различаются только диапазоном строк i и пределом суммирования по j.

Обратите внимание: в шагах 1–2 самый внутренний цикл идёт по $c \in K$ — по **строке** матрицы, то есть по соседним адресам памяти. В отличие от неблочной jki-формы, основной объём работы теперь выполняется с единичным шагом по памяти.

Схема одного шага



Изменяется только блочный столбец K; столбцы правее $k + w$ не трогаются до своей очереди (left-looking). В right-looking-алгоритме, наоборот, после каждого шага обновлялась бы вся правая нижняя подматрица.

5. Вычислительная сложность

Будем считать умножение и сложение отдельными флопами. Для шага с началом блока k:

Шаг	Число флопов	Сумма по всем блокам
1. TRSM $(k \times k) \cdot (k \times w)$	$\approx k^2 w$	$\sum_k k^2 w \approx \int_0^n x^2\,dx = \frac{n^3}{3}$
2. GAXPY $(n-k) \times k \times w$	$2(n-k)\,k\,w$	$2\int_0^n (n-x)\,x\,dx = \frac{n^3}{3}$
3. LU панели $(n-k) \times w$	$\approx (n-k)\,w^2$	$\approx \frac{n^2 r}{2}$

Итого

$$T_{\text{flops}} \approx \frac{2}{3}\,n^3 + O(n^2 r),$$

что совпадает со сложностью обычного метода Гаусса. Важный для следующих этапов вывод: **треугольное решение (шаг 1) и GAXPY (шаг 2) дают примерно поровну работы**, по $n^3/3$ каждый, а LU панели при $r \ll n$ — малая добавка.

Память: n^2 чисел для A (разложение на месте). Для проверки программа хранит ещё копию исходной матрицы A_0 — ещё n^2 чисел.

Выбор размера блока

Размер r — компромисс:

- при малом r внутренний цикл короткий, а доля «неудобного» LU панели и накладных расходов растёт;
- при большом r блок $U_{0:k,K}$ размером $k \times r$ перестаёт помещаться в кэш, а LU панели ($\sim n^2 r/2$ флопов с доступом по столбцу) становится заметным.

В программе $r = 32$: при таком r строка блока занимает $32 \cdot 8 = 256$ байт (4 кэш-линии) и ровно 8 векторов AVX2 (это пригодится на этапе 2). Это значение выбрано по пробным замерам $r \in \{16, 32, 64, 128\}$.

6. Решение системы и проверка

После разложения функция `lu_solve` решает $LUx = b$.

Прямой ход (L с единичной диагональю):

$$y_i = b_i - \sum_{j=0}^{i-1} l_{ij}\,y_j, \qquad i = 0, 1, \ldots, n-1.$$

Обратный ход:

$$x_i = \frac{1}{u_{ii}}\Big(y_i - \sum_{j=i+1}^{n-1} u_{ij}\,x_j\Big), \qquad i = n-1, \ldots, 0.$$

Оба хода стоят $O(n^2)$ и в замер времени не входят.

7. Тестовая матрица и измерение погрешности

Генерация

$$a_{ij} = \xi_{ij} - \tfrac{1}{2}, \quad \xi_{ij} \sim U[0,1]; \qquad a_{ii} \mathrel{+}= \frac{n}{4}.$$

Генератор каждый раз инициализируется `srand(12345)`, поэтому матрица для данного n всегда одна и та же. Сдвиг диагонали делает её доминирующей **в среднем**, но не гарантирует строгого преобладания: математическое ожидание суммы модулей внедиагональных элементов строки

$$\mathbb{E}\sum_{j\neq i}|a_{ij}| = (n-1)\cdot\mathbb{E}|\xi - \tfrac{1}{2}| = \frac{n-1}{4}$$

почти равно диагонали $n/4 \pm 1/2$. Для этих матриц разложение без выбора ведущего элемента на практике проходит, но это свойство конкретных тестовых данных, а не гарантия.

Точное решение

Правая часть — сумма элементов каждой строки:

$$b_i = \sum_j a_{ij} \quad \Longrightarrow \quad x^* = (1, 1, \ldots, 1)^T.$$

Метрики

Программа печатает две величины:

$$\mathrm{err} = \|x - x^*\|_\infty = \max_i |x_i - 1|, \qquad \mathrm{res} = \frac{\|A_0 x - b\|_\infty}{\|b\|_\infty}.$$

- `err` — **прямая** погрешность: насколько найденное решение отличается от точного;
- `res` — относительная **невязка**: насколько хорошо найденный x удовлетворяет исходной системе.

Чего ожидать теоретически

Для метода Гаусса в арифметике с плавающей точкой (машинная точность $\varepsilon = 2^{-53} \approx 1{,}1 \cdot 10^{-16}$) вычисленные множители удовлетворяют

$$\hat{L}\hat{U} = A + \delta A, \qquad |\delta A| \le \gamma_n \, |\hat{L}| \, |\hat{U}|, \qquad \gamma_n = \frac{n\varepsilon}{1 - n\varepsilon},$$

(Higham, теорема 9.3; аналогичная оценка — у Голуба и Ван Лоуна, §3.3). Блочная перестановка порядка суммирования эту оценку не ухудшает. Для прямой погрешности

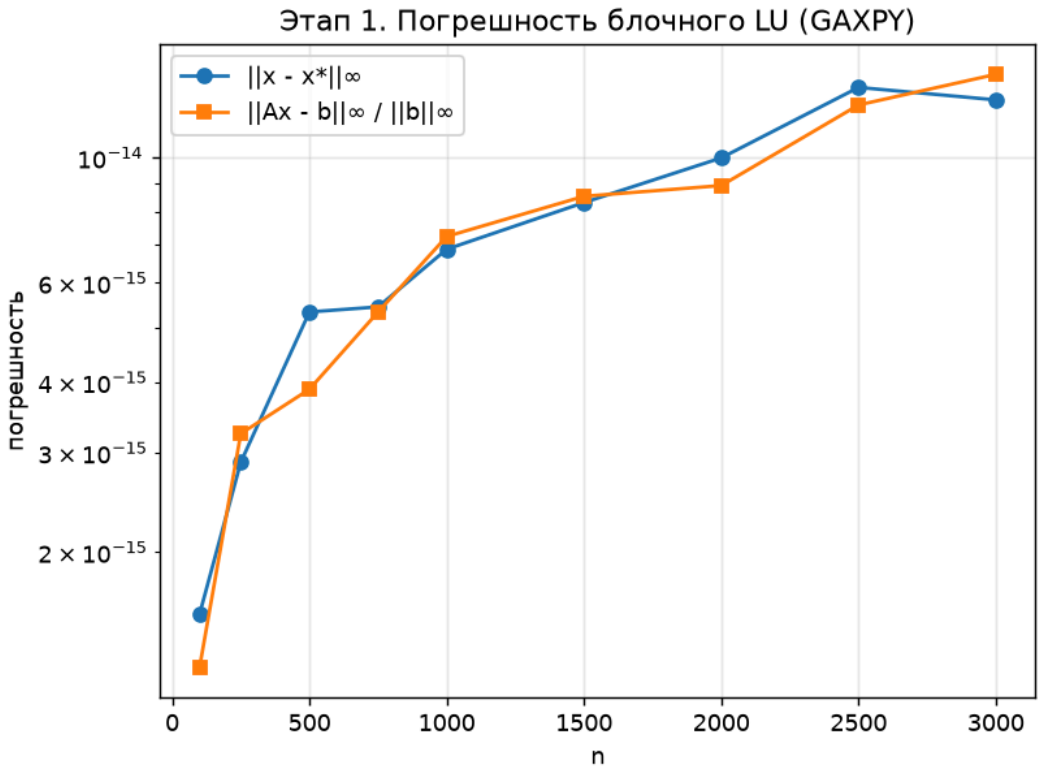
$$\frac{\|x - x^*\|}{\|x^*\|} \lesssim \kappa(A) \cdot \gamma_n \cdot \rho,$$

где $\kappa(A) = \|A\|\|A^{-1}\|$ — число обусловленности, ρ — коэффициент роста элементов. Для матриц с выраженной диагональю $\kappa(A)$ и ρ малы, поэтому **ожидается рост погрешности примерно пропорционально** $n\varepsilon$, но с запасом: оценка γ_n — худший случай, а на практике ошибки округления частично компенсируются (статистически рост ближе к $\sqrt{n}\,\varepsilon$).

Результаты

По stage1_seq.csv:

n	100	250	500	750	1000	1500	2000	2500	3000
err, 10^{-15}	1,55	2,89	5,33	5,44	6,88	8,33	9,99	13,3	12,7
res, 10^{-15}	1,25	3,24	3,89	5,33	7,25	8,54	8,93	12,4	14,1
время, с	0,0002	0,003	0,023	0,080	0,208	0,691	2,147	3,848	6,556



Выводы:

- погрешность **растёт с размерностью**: при увеличении n в 30 раз (с 100 до 3000) `err` растёт примерно в 8 раз — медленнее линейного $n\varepsilon$ и ближе к $\sqrt{n}\,\varepsilon$;
- рост не строго монотонный (при $n = 2500$ ошибка больше, чем при $n = 3000$): каждая матрица своя, и случайный характер ошибок округления заметен;
- во всех случаях ошибка остаётся порядка $10\,\varepsilon$–$100\,\varepsilon$, то есть разложение для этих матриц численно устойчиво;
- время растёт как n^3: от $n = 1000$ до $n = 3000$ — примерно в 31 раз при теоретических 27 (на больших n сказывается кэш).

8. Соответствие коду

Элемент алгоритма	Где в stage1_seq.c
Хранение A по строкам, макрос a_{ij}	`#define a(i, j)`
Размер блока r, число повторов	`R = 32` , `REPS = 3`
Шаги 1–3	`block_lu_gaxpy`
Прямой и обратный ход	`lu_solve`
Генерация A_0, b, замер, метрики	`run`
Список размеров по умолчанию	`main`

Замер времени — `QueryPerformanceCounter` (Windows); из трёх повторов берётся минимальное время. Копирование $A_0 \to A$ выполняется до запуска таймера.

Литература

1. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — §1.1 (GAXPY), §3.2 (формы LU), §3.3 (анализ ошибок).
2. Higham N. J. Accuracy and Stability of Numerical Algorithms. — гл. 9 (LU-разложение), гл. 13 (блочные алгоритмы).
3. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем.

Этап 2. Векторизация блочного LU через GAXPY

Программа: stage2_vec.c. Результаты: stage2_novec.csv, stage2_vec.csv, график stage2_vec_speedup.png.

Алгоритм тот же, что на этапе 1: для каждого блочного столбца $K = [k, k+w)$ выполняются треугольное решение, блочный GAXPY и LU панели. Здесь описано, как код подготовлен к автоматической векторизации компилятором GCC и что из этого получилось.

1. Что такое векторизация

Современный процессор умеет выполнять одну инструкцию сразу над несколькими числами — **SIMD** (Single Instruction, Multiple Data). На процессоре Intel i5-1135G7 доступен набор **AVX2**: регистры по 256 бит, то есть

$$\frac{256 \text{ бит}}{64 \text{ бита на double}} = 4 \text{ числа double в одном регистре.}$$

Дополнительно набор **FMA** (fused multiply-add) выполняет за одну инструкцию

$$d \leftarrow a \cdot b + c$$

с одним округлением вместо двух. Векторная FMA над 4 числами делает 8 флопов.

Автовекторизация — преобразование компилятором скалярного цикла

```
for (c = 0; c < w; c++)
    y[c] -= l * x[c];
```

в цикл, который за одну итерацию обрабатывает 4 элемента:

$$\begin{pmatrix} y_c \\ y_{c+1} \\ y_{c+2} \\ y_{c+3} \end{pmatrix} \leftarrow \begin{pmatrix} y_c \\ y_{c+1} \\ y_{c+2} \\ y_{c+3} \end{pmatrix} - l \cdot \begin{pmatrix} x_c \\ x_{c+1} \\ x_{c+2} \\ x_{c+3} \end{pmatrix}$$ (одна инструкция, например `vfnmadd231pd`).

Теоретическое ускорение такого цикла — до 4 раз за счёт ширины вектора (и ещё до 2 раз, если без FMA умножение и вычитание выполнялись бы отдельно). На практике оно ограничено скоростью подачи данных из памяти.

Условия, при которых компилятор векторизует цикл

1. **Нет зависимостей между итерациями** внутреннего цикла: итерация c не читает то, что пишет итерация $c' \neq c$.
2. **Единичный шаг по памяти**: элементы $x_c, x_{c+1}, \ldots$ лежат подряд — тогда их можно загрузить одной векторной инструкцией.
3. **Известно отсутствие перекрытия** (aliasing) массивов y и x. Если компилятор не может это доказать, он либо отказывается от векторизации, либо вставляет проверку во время выполнения (*versioning*).
4. Простая форма цикла: счётчик с известными границами, без вызовов функций и ветвлений в теле.

2. Ядро алгоритма — операция AXPY по строке

В шагах 1 и 2 блочного алгоритма самая внутренняя операция одинакова:

$$y_K \leftarrow y_K - l \cdot x_K, \qquad y_K, x_K \in \mathbb{R}^w,$$

- в треугольном решении: $u_{i,K} \mathrel{-}= l_{ij}\, u_{j,K}$;
- в GAXPY: $\tilde{a}_{i,K} \mathrel{-}= l_{ij}\, u_{j,K}$.

Это SAXPY над строкой блока длины $w = 32$. Условия 1, 2 и 4 выполнены уже в коде этапа 1. Проблема — условие 3: на этапе 1 и y, и x — части одного массива `A`, и компилятор не знает, что строки i и j не пересекаются.

Как это решено в stage2_vec.c

1. Плотные буферы. Перед шагами 1–2 блок $U_{0:k,K}$ копируется в отдельный буфер `U` размера $k \times w$, а панель $A_{k:n,K}$ — в буфер `P` размера $(n-k) \times w$:

$$\mathtt{U}[i \cdot w + c] = a_{i,\,k+c}, \qquad \mathtt{P}[i \cdot w + c] = a_{k+i,\,k+c}.$$

В буферах шаг строки равен w, а не n: строки блока лежат вплотную, весь блок `U` — один непрерывный участок памяти. Это лучше использует кэш и аппаратную предвыборку (prefetch).

2. Ключевое слово `restrict`. Указатели объявлены как `double *restrict`. Это обещание программиста компилятору: через этот указатель доступна память, к которой нет доступа через другие указатели в той же области видимости. Тогда запись в `ui[c]` не может изменить `uj[c']`, и цикл можно векторизовать.

```
static void gaxpy(int n, int k, int m, int w, const double *restrict A,
                  const double *restrict U, double *restrict P)
{
    for (int i = 0; i < m; i++) {
        const double *restrict li = A + (size_t)(k + i) * n;   /* строка L     */
        double *restrict pi = P + (size_t)i * w;                /* строка панели */
        for (int j = 0; j < k; j++) {
            double l = li[j];                                   /* скаляр       */
            const double *restrict uj = U + (size_t)j * w;      /* строка U     */
            for (int c = 0; c < w; c++)                         /* <- SIMD      */
                pi[c] -= l * uj[c];
        }
    }
}
```

3. Копирование обратно. После разложения панели буферы копируются обратно в `A`. Копирование стоит $O(nw)$ на шаг, то есть $O(n^2)$ всего — пренебрежимо мало по сравнению с $\frac{2}{3}n^3$ флопов.

Почему LU панели не векторизуется

В шаге 3 внутренний цикл идёт по строкам панели i при фиксированном столбце c:

$$p_{ic} \mathrel{-}= p_{ij}\, u_{jc}, \qquad i = j+1, \ldots, m-1,$$

а в буфере `P` (по строкам) соседние i отстоят на w элементов. Это доступ с шагом w (*gather*), который AVX2 эффективно не векторизует. На этот шаг приходится всего $O(n^2 r)$ флопов, поэтому он оставлен скалярным.

3. Сборки и флаги компилятора

Один исходник собирается дважды (build.sh):

Сборка	Флаги	Назначение
`stage2_novec.exe`	`-O2 -fno-tree-vectorize`	база: векторизация запрещена
`stage2_vec.exe`	`-O3 -ftree-vectorize -mavx2 -mfma`	векторизация с AVX2 и FMA

Общие флаги обеих сборок: `-std=gnu99 -msse2 -mfpmath=sse`. Компилятор — 32-битный MinGW GCC 9.2; без `-mfpmath=sse` он использовал бы для `double` устаревший стековый сопроцессор x87, и сравнение было бы нечестным.

Значение флагов:

- `-ftree-vectorize` — включает автовекторизатор циклов (на `-O3` он включён и так; флаг указан явно, чтобы было видно, что именно сравнивается);
- `-fno-tree-vectorize` — явно запрещает его в базовой сборке;
- `-mavx2` — разрешает инструкции AVX2 (256-битные векторы);
- `-mfma` — разрешает инструкции fused multiply-add.

Отчёт компилятора

Флаг `-fopt-info-vec-optimized` печатает, какие циклы векторизованы. Для `stage2_vec.c` GCC 9.2 сообщает:

```
stage2_vec.c:33:13: optimized:  loop versioned for vectorization because of possible aliasing
stage2_vec.c:33:13: optimized: loop vectorized using 32 byte vectors
stage2_vec.c:49:13: optimized: loop vectorized using 32 byte vectors
```

- строка 33 — внутренний цикл треугольного решения (`trsm`): векторизован 32-байтными (AVX) векторами. Несмотря на `restrict`, компилятор вставил проверку перекрытия во время выполнения (versioning): `ui` и `uj` получены из одного буфера `U`, и GCC не смог доказать их независимость статически. Проверка выполняется один раз на входе в цикл и почти ничего не стоит;
- строка 49 — внутренний цикл GAXPY: векторизован без оговорок;
- остальные сообщения относятся к вспомогательным циклам (копирование, генерация матрицы); цикл LU панели в отчёте отсутствует — как и ожидалось.

При $w = 32$ цикл ровно на 8 векторных итераций без «хвоста». Для последнего, укороченного блока ($w < 32$) компилятор выполняет оставшиеся элементы скалярно.

4. Модель ожидаемого ускорения

Пусть T_{scal} — время без векторизации. Разделим его на долю f, приходящуюся на векторизуемые циклы (шаги 1–2), и долю $1 - f$ (LU панели, копирование). Если векторизуемая часть ускоряется в s раз, то по закону Амдала

$$S = \frac{T_{\text{scal}}}{T_{\text{vec}}} = \frac{1}{(1 - f) + f/s}.$$

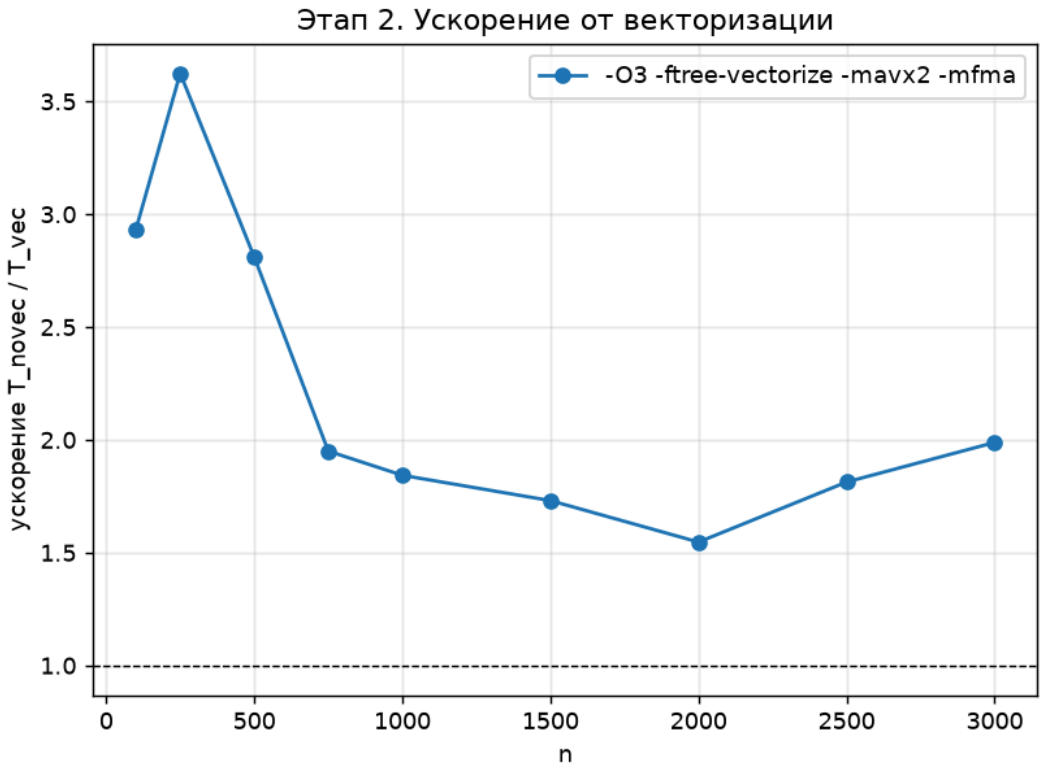
Из раздела 5 этапа 1: шаги 1–2 дают $\frac{2}{3}n^3$ флопов, LU панели — около $\frac{1}{2}n^2 r$, то есть $1 - f \approx \frac{3r}{4n}$; при $n \geq 1000$ это меньше 3%. Значит, ускорение определяется в основном s — тем, насколько быстро работает векторный цикл.

Верхняя граница s по арифметике — 4 (ширина AVX2). Но на одну векторную FMA приходится загрузка 4 элементов `uj` (32 байта) из памяти, а блок U размером $k \times w$ при больших k (до $3000 \cdot 32 \cdot 8 \approx 750$ КБ) не помещается в кэш L1 (48 КБ) и читается из L2. Поэтому при росте n узким местом становится пропускная способность кэша, и реальное s меньше 4.

5. Результаты

$$S(n) = \frac{T_{\text{novec}}(n)}{T_{\text{vec}}(n)}$$

n	100	250	500	750	1000	1500	2000	2500	3000
T_{novec}, с	0,0004	0,0054	0,040	0,104	0,243	0,767	1,715	3,284	5,579
T_{vec}, с	0,0001	0,0015	0,014	0,053	0,132	0,443	1,109	1,811	2,806
S	2,93	3,62	2,81	1,95	1,84	1,73	1,55	1,81	1,99



Выводы:

- при малых n (до 500) ускорение **2,8–3,6** — близко к теоретическому пределу 4: весь блок U лежит в кэше L1/L2, и векторный цикл работает на полной скорости;
- при $n \geq 750$ ускорение падает до **1,6–2,0**: блок U больше не помещается в быстрый кэш, и векторные инструкции простаивают в ожидании данных;
- при $n = 100$ ускорение меньше максимального из-за накладных расходов: блоков всего четыре, и доля скалярного LU панели и копирований велика ($1 - f$ порядка 25%);
- погрешность векторной версии совпадает с этапом 1 до последнего знака, кроме $n = 3000$ ($1{,}243 \cdot 10^{-14}$ против $1{,}266 \cdot 10^{-14}$) и невязки в нескольких точках: FMA выполняет $a \cdot b + c$ с одним округлением вместо двух, поэтому результаты отличаются на уровне машинной точности (векторизуется и сам цикл вычисления невязки, поэтому `res` расходится чаще, чем `err`).

Ограничения эксперимента

Сборки отличаются не только векторизацией: база собрана с `-O2`, векторная — с `-O3`. Уровень `-O3` включает и другие оптимизации (например, более агрессивную развёртку циклов). Поэтому измерено ускорение **сборки в целом**; отдельный вклад SIMD, FMA и `-O3` эти замеры не разделяют. Чтобы его выделить, нужны дополнительные сборки, например `-O3 -fno-tree-vectorize` и `-O3 -ftree-vectorize` без `-mfma`.

6. Отличия от этапа 1

	Этап 1	Этап 2
Данные шагов 1–2	прямо в `A`, шаг строки n	буферы `U`, `P`, шаг строки w
Перекрытие указателей	неизвестно компилятору	`restrict`
LU панели	в `A`, доступ с шагом n	в буфере `P`, доступ с шагом w
Дополнительная память	нет	$2nr$ чисел (буферы `U`, `P`)

Даже без векторизации переход на плотные буферы улучшает работу с кэшем. Поэтому `stage2_novec.exe` и `stage1_seq.exe` при одинаковых флагах `-O2` работают близко по времени, но не одинаково (см. CSV этапов 1 и 2).

Этап 3. OpenMP: блочное LU через GAXPY

Программа: stage3_omp.c. Результаты: `results/stage3_omp_t1.csv` … stage3_omp_t8.csv, графики stage3_omp_speedup_p.png и stage3_omp_speedup_n.png.

Алгоритм — блочный left-looking LU из этапа 1, данные организованы как на этапе 2: блок U и панель P копируются в плотные буферы. Параллельные циклы помечены директивой `#pragma omp parallel for`. **Итерации между потоками распределяет сама OpenMP** — в программе нет ручного деления диапазонов по номеру потока (`omp_get_thread_num`), нет явных барьеров и критических секций.

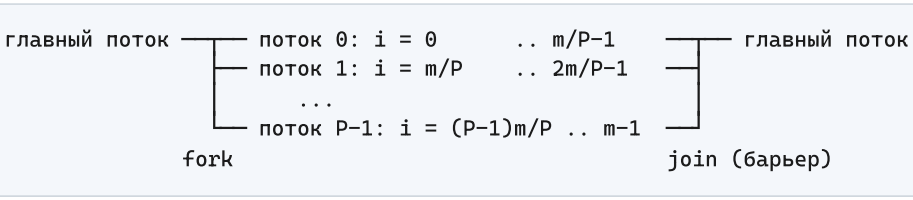
1. Модель OpenMP

OpenMP — стандарт параллельного программирования для систем с **общей памятью**. Программа начинается одним (главным) потоком. Директива

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < m; i++)
    body(i);
```

работает по схеме **fork–join**:

1. *fork* — главный поток запускает команду из P потоков (потоки создаются один раз и затем переиспользуются библиотекой libgomp);
2. итерации цикла $0, \ldots, m-1$ распределяются между потоками;
3. каждый поток выполняет свои итерации;
4. *join* — неявный барьер в конце цикла: главный поток продолжает работу, когда все итерации завершены.



Распределение итераций

`schedule(static)` — итерации делятся на P непрерывных частей почти равного размера: поток t получает

$$\left[\left\lfloor \frac{t\,m}{P} \right\rfloor, \left\lfloor \frac{(t+1)\,m}{P} \right\rfloor\right), \qquad t = 0, \ldots, P-1.$$

Это деление выполняет библиотека OpenMP, а не программа. Статическое расписание подходит, потому что все итерации каждого параллельного цикла имеют одинаковую стоимость.

Переменные

Переменные, объявленные **внутри** тела цикла (`li`, `pi`, `l`, `s`, …), и счётчик параллельного цикла — **локальные** для каждого потока. Массивы `A`, `U`, `P` — **общие**: корректность обеспечивается тем, что разные итерации пишут в разные элементы (см. раздел 2).

Число потоков

Число потоков задаётся при запуске переменной окружения:

```
OMP_NUM_THREADS=4 ./bin/stage3_omp.exe 1000 2000
```

Программа печатает его в столбце `threads` (через `omp_get_max_threads()`). Сборка одна:

```
gcc -std=gnu99 -msse2 -mfpmath=sse -O2 -fno-tree-vectorize -fopenmp stage3_omp.c -o
bin/stage3_omp.exe
```

Флаг `-fopenmp` включает обработку директив и подключает libgomp. Векторизация запрещена (`-fno-tree-vectorize`), как и на этапе 1, — чтобы измерить вклад именно многопоточности. База для ускорения — та же программа с `OMP_NUM_THREADS=1`.

2. Какие циклы можно распараллелить

Директива `parallel for` корректна, только если итерации цикла **независимы**: ни одна итерация не читает и не пишет то, что пишет другая. Различают зависимости:

Тип	Итерация i_1	Итерация $i_2 > i_1$	Пример
истинная (flow, RAW)	пишет	читает	`x[i] = x[i-1] + 1`
анти- (WAR)	читает	пишет	`x[i] = x[i+1]`
по выходу (WAW)	пишет	пишет	`s = x[i]`

Компилятор при `#pragma omp parallel for` **не проверяет** независимость — за неё отвечает программист. Разберём три шага алгоритма.

Шаг 2: GAXPY — строки независимы

$$p_{i,c} \leftarrow p_{i,c} - \sum_{j=0}^{k-1} l_{ij}\, u_{jc}, \qquad i = 0, \dots, m-1,\ c = 0, \dots, w-1.$$

Итерация i внешнего цикла **пишет** только строку i буфера панели (`P[i*w + c]`, $0 \le c < w$) и **читает** строку i матрицы L и весь блок U, которые в этом цикле не меняются. Две итерации $i_1 \neq i_2$ могли бы записать в одну ячейку, только если

$$i_1 w + c_1 = i_2 w + c_2, \quad 0 \le c_1, c_2 < w \quad \Longrightarrow \quad (i_1 - i_2)\, w = c_2 - c_1, \quad |c_2 - c_1| < w,$$

что возможно лишь при $i_1 = i_2$. Значит, внешний цикл по строкам параллелится напрямую:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < m; i++) {
    const double *li = A + (size_t)(k + i) * n;   /* строка L, только чтение */
    double *pi = P + (size_t)i * w;                /* своя строка панели      */
    for (int j = 0; j < k; j++) {
        double l = li[j];
        const double *uj = U + (size_t)j * w;      /* блок U, только чтение   */
        for (int c = 0; c < w; c++)
            pi[c] -= l * uj[c];
    }
}
```

Это половина всей арифметики алгоритма ($\frac{n^3}{3}$ флопов).

Шаг 1: треугольное решение — параллелим по столбцам

$$u_{i,c} \leftarrow u_{i,c} - \sum_{j=0}^{i-1} l_{ij}\, u_{j,c}, \qquad i = 1, \dots, k-1.$$

По строкам i параллелить **нельзя**: строка i читает строки $j < i$, которые записаны предыдущими итерациями (истинная зависимость с расстоянием $i - j \geq 1$). Это рекуррентность прямой подстановки.

Но система $L_{11}U = A$ имеет w правых частей, и **столбцы c независимы**: каждый столбец — отдельная треугольная система $L_{11}u_{:,c} = a_{:,c}$. Поэтому цикл по столбцам вынесен наружу и помечен директивой:

```
#pragma omp parallel for schedule(static)
for (int cb = 0; cb < w; cb += CB) {           /* порция из CB = 8 столбцов     */
    int ce = cb + CB < w ? cb + CB : w;
    for (int i = 1; i < k; i++)                 /* прямая подстановка по строкам */
        for (int j = 0; j < i; j++)
            for (int c = cb; c < ce; c++)
                U[i*w + c] -= A[i*n + j] * U[j*w + c];
}
```

Итерация `cb` пишет только столбцы $[cb, ce)$ блока U, а читает те же столбцы и матрицу L_{11} — пересечений между итерациями нет.

Почему порции по 8 столбцов. Если отдать каждому потоку по одному столбцу, соседние потоки будут писать в соседние элементы одной строки U, то есть в одну кэш-линию (64 байта = 8 чисел `double`). Процессор вынужден передавать такую линию между ядрами при каждой записи — это **ложное разделение** (false sharing), оно сильно замедляет программу. Порция из $CB = 8$ столбцов — ровно одна кэш-линия, а буфер `U` выделен с выравниванием на 64 байта (`_aligned_malloc`). Поэтому каждый поток пишет только в «свои» кэш-линии.

Плата за это — ограниченный параллелизм: при $w = 32$ всего $w/CB = 4$ порции, то есть треугольное решение использует **не больше 4 потоков**. Это совпадает с числом физических ядер процессора i5-1135G7.

Шаг 3: LU панели — параллелим по строкам внутри столбца

Неблочное LU панели P размера $m \times w$ обрабатывает столбцы $c = 0, \dots, w-1$ **последовательно**: столбец c использует готовые столбцы $0, \dots, c-1$. Но внутри столбца строки ниже диагонали независимы:

$$l_{ic} = \frac{1}{u_{cc}} \Big(p_{ic} - \sum_{j=0}^{c-1} l_{ij}\, u_{jc} \Big), \qquad i = c+1, \dots, m-1.$$

Каждая строка i читает свою строку L и столбец U (уже готовый) и пишет только p_{ic}. Поэтому для каждого столбца сначала последовательно досчитывается верхняя часть (u_{ic}, $i \leq c$, — не больше w строк), а затем строки $i > c$ обрабатываются параллельно:

```
for (int c = 0; c < w; c++) {
    for (int i = 1; i <= c; i++)                /* U-часть столбца: последовательно */
        for (int j = 0; j < i; j++)
            P[i*w + c] -= P[i*w + j] * P[j*w + c];
    double p = 1.0 / P[c*w + c];
    #pragma omp parallel for schedule(static)
    for (int i = c + 1; i < m; i++) {           /* L-часть столбца: параллельно      */
        double s = P[i*w + c];
        for (int j = 0; j < c; j++)
            s -= P[i*w + j] * P[j*w + c];
        P[i*w + c] = s * p;
    }
}
```

По сравнению с этапом 1 циклы переставлены: внутренний цикл по j идёт вдоль строки i (соседние адреса), а не вдоль столбца. Порядок вычитаний для каждого элемента прежний ($j = 0, 1, \ldots$), поэтому результат совпадает с последовательной версией побитово.

Копирования

Циклы копирования $A \to U$, $U \to A$, $A \to P$, $P \to A$ тоже помечены `parallel for` по строкам: разные итерации копируют разные строки.

Сводка

Цикл	Параллельно по	Число итераций	Флопы (всего)
шаг 1, TRSM	порциям столбцов	$w/8 = 4$	$\approx n^3/3$
шаг 2, GAXPY	строкам панели	$m = n - k$	$\approx n^3/3$
шаг 3, LU панели	строкам $i > c$ для каждого столбца	$m - c - 1$	$\approx n^2 r/2$
копирования	строкам	k или m	$O(n^2)$

3. Модель ускорения

Обозначим $T_1 \approx \frac{2}{3} n^3 t_c$ — время на одном потоке. Шаг 2 масштабируется на все P потоков, шаг 1 — не больше чем на 4:

$$T_P \approx \frac{n^3}{3} t_c \left(\frac{1}{\min(P,\, 4)} + \frac{1}{P} \right) + T_{\text{fork-join}}, \qquad S(P) = \frac{T_1}{T_P}.$$

P	1	2	3	4	6	8
$S_{\text{модель}}(P)$ без накладных расходов	1,00	2,00	3,00	4,00	4,80	5,33

Модель не учитывает:

- **накладные расходы fork–join.** На каждом блочном шаге запускается $w + 6 = 38$ параллельных областей (32 — в LU панели, по одной на столбец; TRSM; GAXPY; четыре копирования). При $n = 3000$ это около $94 \cdot 38 \approx 3600$ областей, при $n = 500$ — около 600 на разложение длительностью ~30 мс. Каждая область стоит единицы микросекунд на запуск и барьер;
- **общую пропускную способность памяти.** В шаге 1 каждый поток читает весь нижний треугольник L_{11} (до 36 МБ при $k = 3000$ — больше кэша L3), а обновляет только 8 столбцов: на 8 байт данных приходится всего 16 флопов. Четыре потока, одновременно читающие L_{11}, упираются в память;
- **снижение частоты.** При загрузке всех ядер Turbo Boost снижает частоту процессора;

- **число физических ядер — четыре.** При $P > 4$ потоки делят ядра через Hyper-Threading, и вычислительный выигрыш от «лишних» потоков мал.

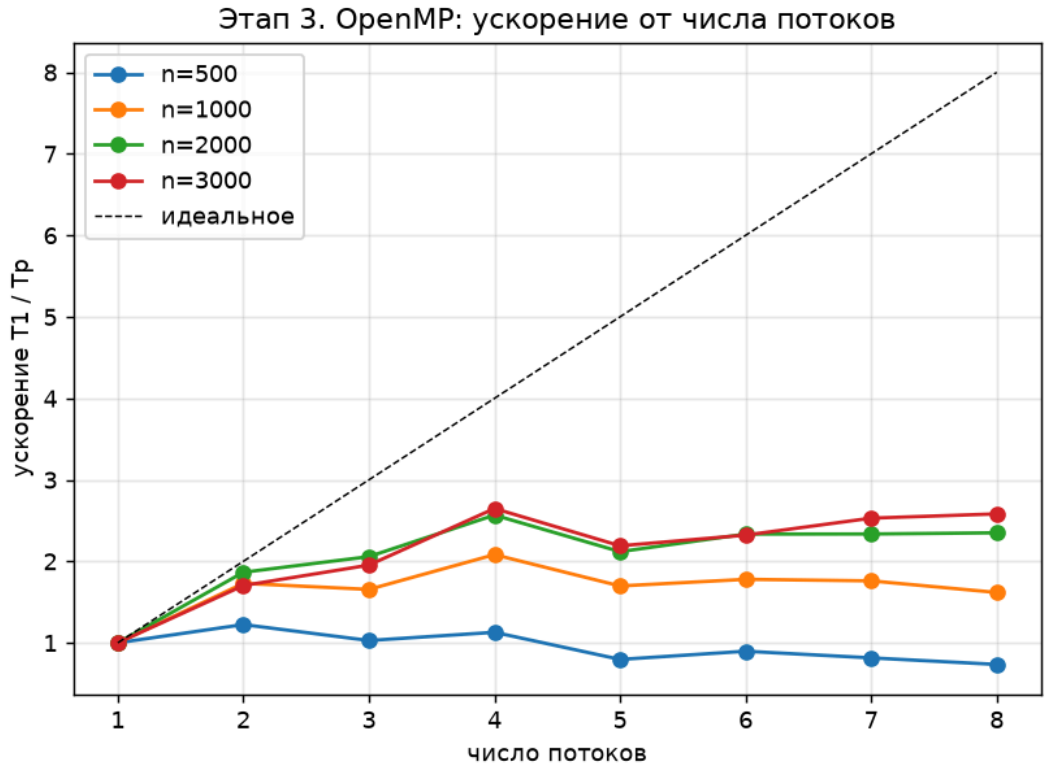
Поэтому реалистичная граница — около $S \approx 4$ при $P = 4$ для больших n.

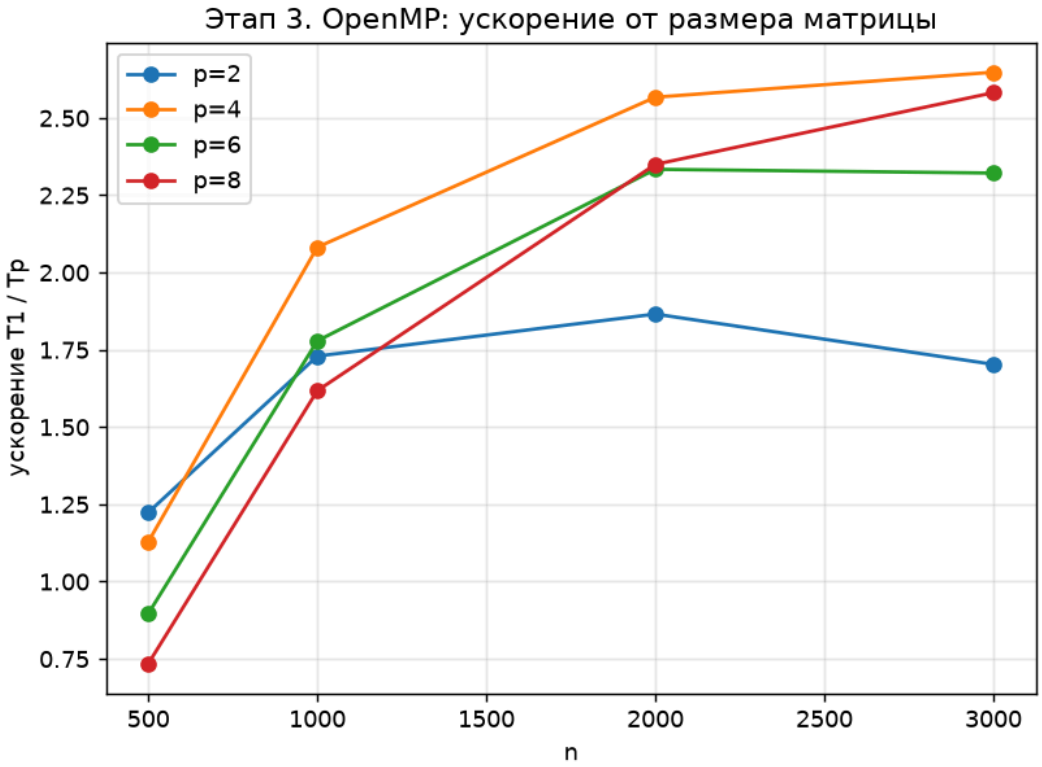
4. Результаты

$$S(P) = \frac{T(\texttt{OMP_NUM_THREADS} = 1)}{T(\texttt{OMP_NUM_THREADS} = P)}$$

$n \setminus P$	1	2	3	4	5	6	7	8
500	1,00	1,22	1,03	1,13	0,80	0,90	0,82	0,73
1000	1,00	1,73	1,66	**2,08**	1,70	1,78	1,76	1,62
2000	1,00	1,87	2,06	**2,57**	2,12	2,33	2,34	2,35
3000	1,00	1,70	1,95	**2,65**	2,19	2,32	2,53	2,58

Время одного потока: 0,029 с ($n = 500$), 0,237 с (1000), 1,848 с (2000), 5,813 с (3000). Эффективность $E = S/P$ при $n = 3000$: 0,85 ($P = 2$), 0,65 ($P = 3$), 0,66 ($P = 4$), 0,32 ($P = 8$).





Выводы

- лучшее ускорение — **2,65** при $n = 3000$ и $P = 4$; для всех $n \geq 1000$ максимум достигается именно при $P = 4$ — числе физических ядер;
- ускорение растёт с размером матрицы: при $P = 4$ — 1,13 → 2,08 → 2,57 → 2,65. Работа растёт как n^3, а число параллельных областей — лишь как n, поэтому доля накладных расходов fork–join падает;
- при $n = 500$ OpenMP почти не даёт выигрыша, а при $P \geq 5$ программа работает **медленнее** однопоточной ($S = 0{,}73$ при $P = 8$): разложение длится ~30 мс, и ~600 запусков параллельных областей с барьерами на перегруженных (Hyper-Threading) ядрах съедают весь выигрыш;
- при $P > 4$ ускорение не растёт: треугольное решение использует не больше 4 потоков, а дополнительные потоки делят физические ядра;
- реальное ускорение (2,65) ниже модели (4,0): шаг 1 ограничен пропускной способностью памяти, а частота процессора снижается при загрузке всех ядер;
- погрешности `err` и `res` для всех P совпадают с последовательной версией: каждый элемент вычисляется одним потоком в том же порядке суммирования.

Сравнение с MPI

OpenMP-версия на этом компьютере работает быстрее MPI-версии (этап 4, максимум 1,96): потоки обращаются к одной общей матрице, без копий у каждого процесса, без переупаковок и коллективных обменов. Кроме того, LU панели здесь распараллелено, а в MPI-версии каждый процесс выполняет его целиком.

Этап 4. MPI-версия блочного LU через GAXPY

Программа: stage4_mpi.c. Результаты: `results/stage4_mpi_p1.csv` … stage4_mpi_p8.csv, графики stage4_mpi_speedup_p.png и stage4_mpi_speedup_n.png.

Алгоритм — блочный left-looking LU из этапа 1. Здесь описано, как работа распределяется между p MPI-процессами и какими сообщениями они обмениваются.

1. Модель MPI

MPI-программа запускается как p независимых процессов с **раздельной памятью**:

```
mpiexec -n 4 bin/stage4_mpi.exe 1000 2000
```

Каждый процесс знает свой номер `rank` $\in \{0, \dots, p-1\}$ и общее число `size` $= p$. Данные между процессами передаются только явными сообщениями. В программе используются коллективные операции:

Операция	Что делает
`MPI_Bcast`	процесс 0 рассылает массив всем
`MPI_Allgatherv`	каждый процесс вносит свой кусок (разной длины), все получают склейку всех кусков
`MPI_Barrier`	синхронизация всех процессов (для замера времени)

Используется реализация MS-MPI; все процессы работают на одном компьютере, и сообщения передаются через общую память.

2. Распределение данных

Вся матрица A хранится у каждого процесса. Процесс 0 генерирует её и рассылает через `MPI_Bcast` (до начала замера времени). После каждого блочного шага все процессы получают одинаковый результат, так что копии A остаются согласованными.

Такой выбор упрощает алгоритм: left-looking-схеме на шаге k нужны **все** предыдущие столбцы L, и при репликации их не нужно пересылать. Цена — память n^2 чисел на каждый процесс (при $n = 3000$ это 72 МБ на процесс).

Распределяется не хранение, а **работа**: на каждом шаге каждый процесс вычисляет свою часть блочного столбца, после чего части собираются у всех.

Разбиение на непрерывные участки

Функция `split(total, parts, cnt, off)` делит N = `total` элементов на p участков:

$$\mathrm{cnt}_q = \left\lfloor \frac{N}{p} \right\rfloor + [\, q < N \bmod p \,], \qquad \mathrm{off}_q = \sum_{s<q} \mathrm{cnt}_s, \qquad q = 0, \dots, p-1.$$

Первые $N \bmod p$ процессов получают на один элемент больше. Например, 32 столбца на 3 процесса: $11 + 11 + 10$.

3. Параллельный алгоритм одного блочного шага

Обозначения, как на этапе 1: $K = [k, k+w)$ — текущий блочный столбец, $m = n - k$ — высота панели.

Шаг 1. Треугольное решение — делим столбцы

Нужно решить

$$L_{11}\, U_{0:k,\,K} = A_{0:k,\,K}.$$

Это w **независимых** треугольных систем — по одной на каждый столбец правой части:

$$L_{11}\, u_{:,c} = a_{:,c}, \qquad c \in K.$$

Значит, столбцы можно распределить между процессами без каких-либо обменов во время решения. Процесс q получает столбцы

$$K_q = [\,k + \mathrm{off}_q,\ k + \mathrm{off}_q + \mathrm{cnt}_q\,), \qquad \sum_q \mathrm{cnt}_q = w,$$

копирует их в плотный буфер `buf` размера $k \times \mathrm{cnt}_q$ и решает прямой подстановкой в форме GAXPY по строкам:

$$u_{i,K_q} \leftarrow u_{i,K_q} - \sum_{j=0}^{i-1} l_{ij}\, u_{j,K_q}, \qquad i = 1, \dots, k-1.$$

Тот же приём использует OpenMP-версия на этапе 3: зависимость по строкам i остаётся, а независимые столбцы делятся между исполнителями.

Сбор. `MPI_Allgatherv` собирает куски всех процессов в буфер `U`. Каждый кусок — это блок $k \times \mathrm{cnt}_q$, записанный по строкам, поэтому после сбора данные лежат «по процессам», а не по строкам:

```
U после Allgatherv:  [ блок процесса 0 (k x cnt0) | блок процесса 1 (k x cnt1) | ... ]
нужно:               строка 0: [cnt0 | cnt1 | ...], строка 1: [cnt0 | cnt1 | ...], ...
```

Поэтому выполняется переупаковка: столбцы каждого процесса записываются на свои места в $A_{0:k,K}$, а затем из A формируется плотный блок `U` размера $k \times w$ для следующего шага.

Шаг 2. GAXPY — делим строки

$$\tilde{A}_{k:n,\,K} = A_{k:n,\,K} - L_{k:n,\,0:k}\, U_{0:k,\,K}.$$

Строки панели независимы (см. анализ зависимостей), поэтому m строк делятся между процессами: процесс q обрабатывает строки

$$I_q = [\,\mathrm{off}_q,\ \mathrm{off}_q + \mathrm{cnt}_q\,) \quad \text{(в нумерации панели)},$$

записывая результат в свои строки буфера панели `P`. Каждому процессу нужен весь блок $U_{0:k,K}$ (он уже есть у всех после шага 1) и свои строки L (они есть в копии A).

Сбор. Строки `P` хранятся по строкам и подряд, поэтому участок процесса q — непрерывный кусок `P[off_q * w ... (off_q + cnt_q) * w)`. Сбор выполняется `MPI_Allgatherv` с `MPI_IN_PLACE`: каждый процесс уже записал свою часть на нужное место, и операция лишь заполняет остальные части.

Шаг 3. LU панели — избыточно на каждом процессе

После шага 2 у каждого процесса есть полная панель $\tilde{A}_{k:n,K}$ в буфере `P`. Её неблочное LU (стоимостью $\approx (n-k)\,w^2$ флопов) **каждый процесс выполняет сам** и получает одинаковый результат. Распределять этот шаг

невыгодно: столбцы панели обрабатываются строго последовательно, и на каждый из w столбцов понадобился бы отдельный обмен.

Первый блок

При $k = 0$ слева нет готовых столбцов: шаги 1 и 2 пропускаются, и сразу выполняется LU первой панели.

Псевдокод

```
для k = 0, r, 2r, ... < n:
    w = min(r, n-k);  m = n-k
    если k > 0:
        # шаг 1: столбцы
        (cnt, off) = split(w, p)
        buf  = A(0:k, мои столбцы)
        buf  = L11^{-1} buf                          # локально, без обменов
        U    = Allgatherv(buf)                       # обмен 1: k*w чисел
        переупаковать U -> A(0:k, K) -> U (k x w)
        # шаг 2: строки
        (cnt, off) = split(m, p)
        P(мои строки) = A(k+мои строки, K) - L(мои строки, 0:k) * U
        Allgatherv(IN_PLACE, P)                      # обмен 2: m*w чисел
    иначе:
        P = A(0:n, 0:w)
    LU(P)                                            # на каждом процессе
    A(k:n, K) = P
```

4. Анализ стоимости

Обозначим t_c — время одного флопа, α — задержку (латентность) сообщения, β — время передачи одного байта. Для `Allgatherv` с итоговым объёмом V байт в модели «кольца»:

$$T_{\text{allgather}}(V) \approx (p-1)\,\alpha + \frac{p-1}{p}\,V\beta.$$

Вычисления

Шаг	Работа одного процесса на шаге k	Сумма по шагам
1. TRSM	$k^2 \cdot \text{cnt}_q \approx k^2 w/p$	$\approx \dfrac{n^3}{3p}$
2. GAXPY	$2k \cdot \text{cnt}_q \cdot w \approx 2kmw/p$	$\approx \dfrac{n^3}{3p}$
3. LU панели	$\approx mw^2$ (не делится)	$\approx \dfrac{n^2 r}{2}$
переупаковки	$O(nw)$ (не делится)	$O(n^2)$

Обмены

На каждом шаге (их n/r) — два `Allgatherv` объёмом kw и $(n-k)w$ чисел, вместе nw чисел, то есть $8nw$ байт. За всё разложение:

$$T_{\text{comm}} \approx \frac{2n}{r}\,(p-1)\,\alpha \;+\; \frac{p-1}{p}\,8n^2\beta.$$

Итоговая модель

$$T_p \approx \underbrace{\frac{2n^3}{3p}\,t_c}_{\text{делится}} + \underbrace{\frac{n^2 r}{2}\,t_c + O(n^2)}_{\text{не делится}} + \underbrace{\frac{2n}{r}(p-1)\alpha + \frac{p-1}{p}\,8n^2\beta}_{\text{обмены}},$$

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}.$$

Главный член $\frac{2n^3}{3p}$ растёт как n^3, а обмены и неделимая часть — как n^2. Поэтому **при росте n ускорение должно приближаться к p**, а при малых n и больших p обмены и синхронизации преобладают.

Балансировка нагрузки

Столбцы треугольного решения делятся на p частей с округлением вверх у части процессов. Самый загруженный процесс получает $\lceil w/p \rceil$ столбцов вместо w/p:

p	2	3	4	5	6	7	8
$\lceil 32/p \rceil$	16	11	8	7	6	5	4
потеря на шаге 1, %	0	3	0	9	13	9	0

Строки GAXPY (m до 3000) делятся почти идеально.

Что модель не учитывает

Модель считает, что флоп стоит одинаково независимо от числа процессов. На одном ноутбуке это не так:

- **Пропускная способность памяти общая.** В шаге 1 каждый процесс читает **весь** нижний треугольник L_{11} (до $\frac{k^2}{2} \cdot 8$ байт, 36 МБ при $k = 3000$ — больше кэша L3), а обновляет лишь w/p столбцов. На каждый прочитанный элемент L приходится $2w/p$ флопов: при $p = 4$ это 16 флопов на 8 байт. С ростом p объём чтения L_{11} всеми процессами вместе растёт как $p \cdot \frac{k^2}{2}$, и шаг 1 упирается в память.
- **Частота процессора.** При загрузке всех ядер Turbo Boost снижает частоту.
- **Физических ядер четыре.** При $p > 4$ два процесса делят одно ядро (Hyper-Threading); к тому же MS-MPI в коллективных операциях ожидает данные активным опросом, отнимая у соседнего процесса время ядра.

5. Замер времени

```
MPI_Bcast(A, n * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);   /* вне замера */
MPI_Barrier(MPI_COMM_WORLD);
double t = MPI_Wtime();
block_lu_gaxpy_mpi(n, R, A);                          /* все обмены внутри */
MPI_Barrier(MPI_COMM_WORLD);                          /* ждём самого медленного */
t = MPI_Wtime() - t;
```

В замер входят вычисления, оба `Allgatherv` на каждом шаге, переупаковки, выделение буферов и завершающий барьер. Начальная рассылка матрицы не входит. Из трёх повторов берётся минимум; печатает результат процесс 0, он же проверяет решение (`err` , `res` — как на этапе 1).

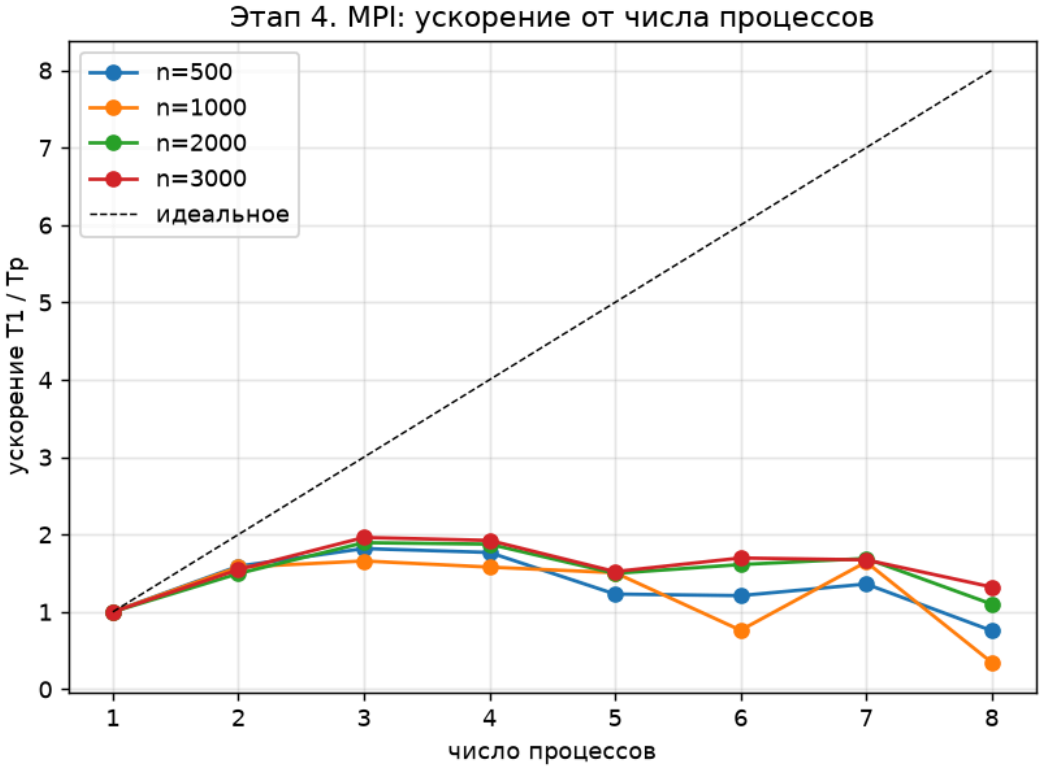
6. Результаты

Ускорение от числа процессов

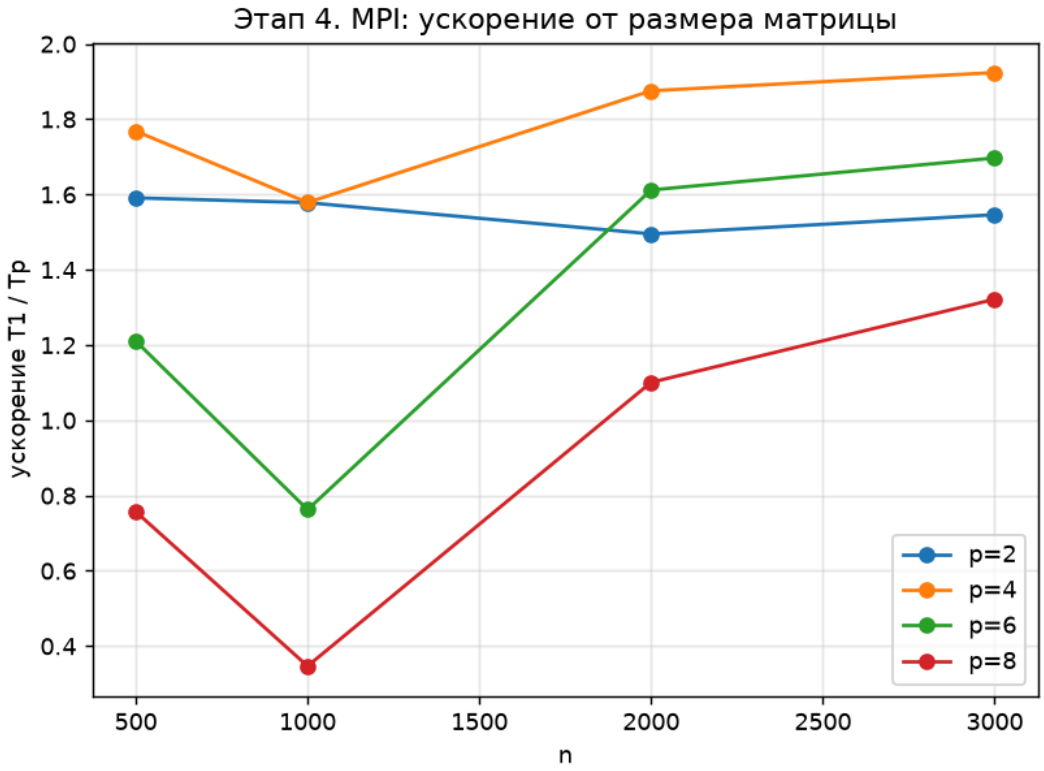
$$S(p) = \frac{T_1}{T_p}$$

$n \setminus p$	1	2	3	4	5	6	7	8
500	1,00	1,59	1,82	1,77	1,23	1,21	1,36	0,76
1000	1,00	1,58	1,66	1,58	1,51	0,76	1,64	0,35
2000	1,00	1,50	1,90	1,88	1,50	1,61	1,69	1,10
3000	1,00	1,55	**1,96**	1,92	1,52	1,70	1,67	1,32

Время одного процесса T_1: 0,032 с ($n = 500$), 0,236 с (1000), 1,817 с (2000), 5,893 с (3000). Эффективность $E = S/p$ при $n = 3000$: 0,78 ($p = 2$), 0,65 ($p = 3$), 0,48 ($p = 4$), 0,17 ($p = 8$).



Ускорение от размера матрицы



Выводы

- лучшее ускорение — **1,96** при $n = 3000$, $p = 3$; при $p = 3$–4 ускорение стабильно 1,7–1,9 для $n \geq 2000$. Это ниже, чем у OpenMP-версии (этап 3, до 2,65 при $P = 4$): там потоки работают с одной общей матрицей без коллективных обменов и переупаковок, а LU панели тоже распараллелено;
- до $p = 3$–4 (число физических ядер) ускорение растёт, дальше — нет: при $p = 5$–7 процессы делят ядра, а при $p = 8$ ускорение падает, особенно на малых матрицах (0,35 при $n = 1000$): доля обменов и ожидания в барьерах становится больше полезной работы;
- с ростом n ускорение при фиксированном p в целом растёт (например, при $p = 8$: 0,76 → 0,35 → 1,10 → 1,32), как предсказывает модель: вычисления $\sim n^3$, обмены $\sim n^2$. Зависимость немонотонна при $n = 1000$: на малых размерах сильнее заметен шум;
- реальное ускорение заметно ниже модели $S \approx p$: основное ограничение здесь — общая память и частота процессора одного ноутбука, а не объём передаваемых данных (за всё разложение каждый процесс получает около $8n^2$ байт — 72 МБ при $n = 3000$, что через общую память передаётся за десятки миллисекунд);
- погрешности `err` и `res` для всех p совпадают с последовательной версией: каждый элемент вычисляется одним процессом в том же порядке суммирования.

Как улучшить масштабирование

- распределить хранение матрицы (например, блочно-циклически по столбцам), чтобы уменьшить память и объём чтения L_{11} на процесс;
- совмещать обмены с вычислениями (неблокирующие `MPI_Iallgatherv`): пока собирается панель шага k, начинать треугольное решение шага $k + r$;
- запускать на кластере или нескольких виртуальных машинах, где у каждого процесса собственная память и ядро.